

UFW + iptables: Complete Cheatsheet & 20 Practical Labs

Beginner to Advanced Mastery

PART 1: COMPLETE REFERENCE GUIDE

1. Firewall Fundamentals

What is a Firewall?

A firewall is a security system that monitors and controls incoming/outgoing network traffic based on predetermined security rules.

Three-Way Firewall Logic

```
INPUT Chain:  [Incoming Traffic] → Accept/Reject/Drop
OUTPUT Chain: [Outgoing Traffic] → Accept/Reject/Drop
FORWARD Chain: [Packet Routing] → Accept/Reject/Drop
```

Key Difference: UFW vs iptables

Aspect	UFW	iptables
Complexity	Simple (user-friendly)	Complex (powerful)
Learning Curve	Beginner	Intermediate to Advanced
Use Case	Quick deployment	Fine-tuned control
Persistence	Automatic	Requires configuration
Rule Format	Simple syntax	Complex syntax
Default	Disabled	N/A (runs always)

2. UFW Complete Reference Guide

Installation

```
bash
```

```
# Install UFW
sudo apt update
sudo apt install -y ufw

# Verify installation
ufw version

# Check current status
sudo ufw status
```

Basic Firewall Control

Enable/Disable Firewall

```
bash

# Enable UFW
sudo ufw enable

# Disable UFW
sudo ufw disable

# Reload configuration
sudo ufw reload

# Reset to default (DESTRUCTIVE - removes all rules)
sudo ufw reset
```

Default Policies

```
bash

# Set default INPUT policy (most important)
sudo ufw default deny incoming      # Drop all incoming by default
sudo ufw default allow incoming     # Allow all incoming (rarely used)

# Set default OUTPUT policy
sudo ufw default allow outgoing     # Allow all outgoing (default)
sudo ufw default deny outgoing      # Drop all outgoing (restrictive)

# Set default FORWARD policy
sudo ufw default deny forward       # Drop forwarded packets
sudo ufw default allow forward      # Allow forwarded packets
```

Adding Rules

Allow Rules

```
bash

# Allow port by number
sudo ufw allow 22                # Allow port 22 (TCP and UDP)
sudo ufw allow 22/tcp            # Allow port 22 TCP only
sudo ufw allow 22/udp            # Allow port 22 UDP only

# Allow by service name
sudo ufw allow ssh                # Equivalent to port 22/tcp
sudo ufw allow http              # Port 80/tcp
sudo ufw allow https             # Port 443/tcp
sudo ufw allow smtp              # Port 25/tcp

# Allow from specific IP
sudo ufw allow from 192.168.1.100 # All traffic from specific IP
sudo ufw allow from 192.168.1.100 to any port 22 # SSH from specific IP

# Allow IP range (CIDR notation)
sudo ufw allow from 192.168.1.0/24 # All traffic from subnet
sudo ufw allow from 10.0.0.0/8 to any port 443 # HTTPS from private network
```

Deny Rules

```
bash

# Deny port
sudo ufw deny 23                # Deny Telnet

# Deny from specific IP
sudo ufw deny from 192.168.1.50 # Deny all traffic from IP

# Deny specific port from IP
sudo ufw deny from 203.0.113.0 to any port 22 # Deny SSH from external IP
```

Reject Rules (send explicit rejection)

```
bash
```

```
# Reject instead of drop (host receives rejection notice)
sudo ufw reject 139 # Reject NetBIOS
sudo ufw reject from 192.168.1.100 # Reject all from IP
```

Removing Rules

Three methods to delete rules:

```
bash

# Method 1: Delete by rule (easiest to remember)
sudo ufw delete allow 22
sudo ufw delete allow from 192.168.1.100

# Method 2: Delete by line number (useful for multiple similar rules)
sudo ufw status numbered # Show rules with numbers
sudo ufw delete 1 # Delete rule #1

# Method 3: Reset all rules
sudo ufw reset # Warning: removes everything
```

Viewing Rules

Check Status

```
bash

# Simple status
sudo ufw status

# Detailed status
sudo ufw status verbose

# Show rules with line numbers (for deletion)
sudo ufw status numbered

# Show all rules (including disabled)
sudo ufw show added
```

Advanced UFW Features

Logging

```
bash
```

```
# Enable logging
sudo ufw logging on

# Set logging level (low, medium, high)
sudo ufw logging medium

# Disable logging
sudo ufw logging off

# View UFW logs
sudo tail -f /var/log/ufw.log

# View blocked packets
sudo grep "UFW BLOCK" /var/log/ufw.log
```

Rate Limiting (DDoS Protection)

```
bash

# Rate limit port 22 to 5 connections per minute
sudo ufw limit 22/tcp

# Rate limit with burst protection
sudo ufw limit 22/tcp comment 'SSH rate limit'

# Difference between allow and limit:
# allow    = unlimited connections
# limit    = max 5 connections/minute after burst (10)
```

Named Profiles

```
bash

# Using service profiles
sudo ufw allow SSH

# Using custom descriptions
sudo ufw allow 3306 comment 'MySQL from 192.168.1.0/24'

# Viewing with descriptions
sudo ufw status verbose
```

Direction-Specific Rules

```
bash
```

```
# Only on specific network interface
```

```
sudo ufw allow in on eth0 to any port 22
```

```
sudo ufw allow out on eth1 from any to any port 443
```

```
# Both directions
```

```
sudo ufw allow in 22
```

```
sudo ufw allow out 22
```

3. iptables Complete Reference Guide

Prerequisites

```
bash
```

```
# Install iptables (usually pre-installed)
```

```
sudo apt install -y iptables iptables-persistent
```

```
# For Debian/Ubuntu persistence
```

```
sudo apt install -y netfilter-persistent
```

Understanding iptables Structure

Three Main Tables

```
nat      - Network Address Translation (IP masquerading, port forwarding)
```

```
filter   - Packet filtering (allow/drop/reject)
```

```
mangle   - Packet modification (TTL, TOS marking)
```

Three Main Chains

```
INPUT    - Incoming packets to your machine
```

```
OUTPUT   - Outgoing packets from your machine
```

```
FORWARD  - Packets passing through (routing)
```

Basic iptables Syntax

General Command Structure

```
bash
```

```
sudo iptables [-t table] [-A/-I/-D/-R] [chain] [options] [-j target]
```

```
-t      = Specify table (default: filter)
-A      = Append rule to end of chain
-I      = Insert rule at beginning (or specific position)
-D      = Delete rule
-R      = Replace rule
-L      = List rules
-F      = Flush (delete all rules in chain)
-P      = Set default policy
-j      = Jump to target (ACCEPT, DROP, REJECT, etc.)
```

Setting Default Policies

```
bash
```

```
# View current policies
```

```
sudo iptables -L | grep "policy"
```

```
# Set default policies (most important for security)
```

```
sudo iptables -P INPUT DROP      # Drop all incoming
```

```
sudo iptables -P FORWARD DROP   # Drop forwarded packets
```

```
sudo iptables -P OUTPUT ACCEPT   # Allow all outgoing
```

```
# Reset to permissive (temporary testing)
```

```
sudo iptables -P INPUT ACCEPT
```

```
sudo iptables -P OUTPUT ACCEPT
```

```
sudo iptables -P FORWARD ACCEPT
```

Adding Rules

Basic Allow/Deny

```
bash
```

```
# Allow specific port (TCP)
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow specific port (UDP)
sudo iptables -A INPUT -p udp --dport 53 -j ACCEPT

# Allow both TCP and UDP
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p udp --dport 80 -j ACCEPT

# Allow from specific IP
sudo iptables -A INPUT -s 192.168.1.100 -j ACCEPT

# Allow from IP range
sudo iptables -A INPUT -s 192.168.1.0/24 -j ACCEPT

# Allow to specific destination IP
sudo iptables -A INPUT -d 10.0.0.1 -j ACCEPT

# Deny instead of allow
sudo iptables -A INPUT -p tcp --dport 23 -j DROP

# Reject with error message
sudo iptables -A INPUT -p tcp --dport 139 -j REJECT --reject-with icmp-host-pro
```

Stateful Rules (Connection Tracking)

```
bash

# Allow established and related connections (CRITICAL!)
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Alternative syntax (newer systems)
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# Allow new connections only
sudo iptables -A INPUT -m state --state NEW -j ACCEPT

# Drop invalid connections
sudo iptables -A INPUT -m state --state INVALID -j DROP
```

Loopback Interface (Required)


```
bash
```

```
# Allow loopback (required for system stability)
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT
```

ICMP (Ping)

```
bash
```

```
# Allow ping (ICMP echo request)
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
sudo iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT

# Block all ICMP (disable ping)
sudo iptables -A INPUT -p icmp -j DROP
```

Viewing Rules

List Rules

```
bash
```

```
# List all rules with line numbers
sudo iptables -L -n --line-numbers

# List specific chain
sudo iptables -L INPUT -n --line-numbers

# Verbose output
sudo iptables -L -v -n

# View in command format (easy to replicate)
sudo iptables -L -n -v --line-numbers

# Count packets matched by each rule
sudo iptables -L -n -v
```

Saving and Viewing State

```
bash
```

```
# Save current rules to file
sudo iptables-save > /tmp/iptables-backup.rules

# View saved rules
cat /tmp/iptables-backup.rules

# Restore from file
sudo iptables-restore < /tmp/iptables-backup.rules
```

Deleting Rules

Three Methods

```
bash

# Method 1: Specify exact rule
sudo iptables -D INPUT -p tcp --dport 22 -j ACCEPT

# Method 2: Delete by line number (from iptables -L -n --line-numbers)
sudo iptables -D INPUT 1      # Delete rule #1 in INPUT chain

# Method 3: Delete entire chain
sudo iptables -F INPUT        # Flush (delete all rules) in INPUT chain
sudo iptables -F              # Flush all chains
```

Advanced iptables Features

Rate Limiting (DDoS Protection)

```
bash

# Limit connections to 5 per minute
sudo iptables -A INPUT -p tcp --dport 22 -m limit --limit 5/minute -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j DROP

# Limit with burst (allow burst then rate limit)
sudo iptables -A INPUT -p tcp --dport 80 -m limit --limit 25/hour --limit-burst 100 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

Port Forwarding (NAT)

```
bash
```

```
# Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1

# Forward port 8080 to 80
sudo iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT --to-port 80

# Forward to another machine
sudo iptables -t nat -A PREROUTING -p tcp --dport 22 -j DNAT --to-destination 192.168.1.100
sudo iptables -A FORWARD -p tcp -d 192.168.1.100 --dport 22 -j ACCEPT
```

Masquerading (IP Spoofing Prevention)

```
bash

# Enable masquerading for outgoing traffic
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# For specific subnet
sudo iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth0 -j MASQUERADE
```

Multiple Ports

```
bash

# Allow multiple ports at once
sudo iptables -A INPUT -p tcp -m multiport --dports 22,80,443 -j ACCEPT

# Allow port range
sudo iptables -A INPUT -p tcp --dport 8000:9000 -j ACCEPT
```

Logging

```
bash
```

```
# Log dropped packets
sudo iptables -A INPUT -m limit --limit 5/min -j LOG --log-prefix "IPT_DROP: "
sudo iptables -A INPUT -j DROP

# View logs
sudo tail -f /var/log/kern.log | grep IPT_DROP

# Log before drop
sudo iptables -I INPUT 1 -p tcp --dport 23 -j LOG --log-prefix "SSH_ATTEMPT: "
```

Connection Timeout Rules

```
bash

# Close connections idle for 5 minutes (600 seconds)
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED -m recent --set
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED -m recent --update --
```

Persistence (Making Rules Permanent)

UFW (Automatic)

```
bash

# UFW saves rules automatically
sudo ufw enable
# Rules persist after reboot
```

iptables (Manual Methods)

Method 1: Using iptables-persistent

```
bash
```

```
# Install iptables-persistent
sudo apt install -y iptables-persistent netfilter-persistent

# Save current rules
sudo iptables-save | sudo tee /etc/iptables/rules.v4
sudo ip6tables-save | sudo tee /etc/iptables/rules.v6

# Restore automatically on boot
sudo systemctl enable netfilter-persistent
```

Method 2: Using startup script

```
bash

# Create script
sudo nano /etc/network/if-pre-up.d/iptables

# Add content:
#!/bin/bash
/sbin/iptables-restore < /etc/iptables/rules.v4
/sbin/ip6tables-restore < /etc/iptables/rules.v6

# Make executable
sudo chmod +x /etc/network/if-pre-up.d/iptables
```

Method 3: Using systemd service

```
bash
```

```
# Create service file
sudo nano /etc/systemd/system/iptables-load.service

# Add:
[Unit]
Description=Load iptables rules
After=network.target

[Service]
Type=oneshot
ExecStart=/sbin/iptables-restore /etc/iptables/rules.v4
ExecStart=/sbin/ip6tables-restore /etc/iptables/rules.v6
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target

# Enable and test
sudo systemctl enable iptables-load
sudo systemctl start iptables-load
```

4. UFW + iptables Interaction

How UFW Uses iptables

```
UFW Rules ↓
↓
Converted to iptables rules ↓
↓
Applied to iptables chains (INPUT, OUTPUT, FORWARD) ↓
↓
Kernel packet filtering
```

When to Use Each

Use UFW When:

- ☒ Setting up basic firewall rules quickly
- ☒ Managing simple allow/deny rules
- ☒ Working in server/production environments
- ☒ Need automatic persistence

-  Want simple syntax for team

Use iptables When:

-  Need fine-grained control
-  Doing NAT and port forwarding
-  Complex rule combinations
-  Packet modification (mangle table)
-  Learning firewall internals
-  Writing automated scripts

Use Both When:

-  UFW for main rules + iptables for custom rules
-  UFW for management + iptables for advanced features
-  Learning progression (start UFW → transition to iptables)

Conflict Prevention

```
bash

# Check if UFW is enabled
sudo ufw status

# If UFW is enabled, adding iptables rules directly can cause conflicts
# Solution: Disable UFW before adding custom iptables rules
sudo ufw disable

# Or: Add iptables rules before enabling UFW
sudo iptables -A INPUT -p tcp --dport 3306 -j ACCEPT
sudo iptables-save | sudo tee /etc/iptables/rules.v4
sudo ufw enable
```

PART 2: 20 PRACTICAL LABS (BEGINNER TO ADVANCED)

Lab Setup Instructions

Prerequisites for All Labs

```
bash
```

```
# 1. Have Kali Linux or Ubuntu server running
# 2. SSH access or terminal access
# 3. sudo privileges
# 4. Two machines (recommend: host + VirtualBox VM)

# Quick setup:
sudo apt update && sudo apt upgrade -y
sudo apt install -y ufw iptables iptables-persistent netcat-openbsd nmap

# Verify setup:
sudo ufw status
sudo iptables -L
```

Important Safety Rules

```
bash

⚠ ALWAYS BACKUP BEFORE CHANGES:
sudo iptables-save > /tmp/iptables-backup-lab-X.rules

⚠ NEVER LOSE SSH ACCESS:
Always allow port 22 before setting default deny!

⚠ TEST BEFORE DEPLOYMENT:
Use test VM, not production server!

⚠ SAVE YOUR WORK:
Document all rules for each lab in notes
```

LEVEL 1: UFW BASICS (Labs 1-5)

LAB 1: UFW Installation and Basic Configuration

Objective: Install UFW and configure basic firewall policies

Difficulty: ★ Beginner

Time: 10 minutes

Prerequisites:

```
bash
```



```
# Verify UFW not already installed
sudo ufw status
```

Lab Steps:

```
bash

# Step 1: Install UFW
sudo apt update
sudo apt install -y ufw

# Step 2: Verify installation
ufw version

# Step 3: Check current status (should be inactive)
sudo ufw status

# Step 4: Set default policies (DROP incoming, ALLOW outgoing)
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw default deny forward

# Step 5: Verify policies set correctly
sudo ufw status verbose

# Step 6: Enable UFW
sudo ufw enable
# Confirm with 'y' when prompted

# Step 7: Verify UFW is active
sudo ufw status
sudo ufw status verbose
```

Validation:

```
bash
```

```
# Check UFW is enabled
sudo systemctl is-active ufw
# Output: active

# Verify default policies
sudo ufw status verbose
# Should show:
# Default: deny (incoming), allow (outgoing), deny (forward)
```

Expected Output:

```
Status: active

To Action From
--
-- deny Anywhere
-- allow Anywhere (outgoing)
```

Common Mistakes:

- ❌ Enabling UFW without allowing SSH (you'll lock yourself out!)
- ❌ Not verifying status after each step

Solution:

```
bash

# If locked out, reboot into single-user mode or:
# From host machine: VirtualBox → Tools → Properties
# Reset network to default
```

Post-Lab Questions:

1. What does "default deny incoming" mean?
2. Why do we allow outgoing by default?
3. How would you disable UFW if needed?

LAB 2: Allow and Deny Basic Ports

Objective: Learn to allow and deny specific ports using UFW

Difficulty: ★ Beginner

Time: 15 minutes

Prerequisites:

```
bash

# Lab 1 completed with UFW enabled
sudo ufw status
# Must show: active
```

Lab Steps:

```
bash

# Step 1: IMPORTANT - Allow SSH before any restrictions
sudo ufw allow 22/tcp
sudo ufw allow ssh # Alternative

# Step 2: Verify SSH rule added
sudo ufw status numbered

# Step 3: Allow HTTP (port 80)
sudo ufw allow 80/tcp
sudo ufw allow http

# Step 4: Allow HTTPS (port 443)
sudo ufw allow 443/tcp
sudo ufw allow https

# Step 5: Allow DNS (port 53)
sudo ufw allow 53/udp
sudo ufw allow 53/tcp

# Step 6: View all allowed rules
sudo ufw status verbose

# Step 7: Deny Telnet (port 23)
sudo ufw deny 23/tcp

# Step 8: Verify both allow and deny rules
sudo ufw status numbered
```

Validation:

```
bash
```

```
# Test SSH connection (should work)
```

```
ssh user@localhost
```

```
# Should connect successfully
```

```
# Test HTTP (from another machine)
```

```
curl http://localhost:80
```

```
# May fail if no web server, but firewall shouldn't block
```

```
# Test denied port (should fail)
```

```
nc -zv localhost 23
```

```
# Should show: Connection refused
```

Expected Output:

```
Status: active
```

To	Action	From
--	-----	----
22/tcp	ALLOW	Anywhere
80/tcp	ALLOW	Anywhere
443/tcp	ALLOW	Anywhere
53/tcp	ALLOW	Anywhere
53/udp	ALLOW	Anywhere
23/tcp	DENY	Anywhere

Common Mistakes:

-  Forgetting to allow SSH first
-  Mixing TCP and UDP (different protocols!)
-  Not verifying each rule

Post-Lab Questions:

1. What's the difference between allowing by port number vs service name?
2. Why do we need both TCP and UDP for DNS?
3. How would you check which rules are actually active?

Challenge Exercise:

```
bash
```

```
# Create these rules:
# - Allow MySQL from only your subnet (192.168.1.0/24)
# - Deny BitTorrent (6881-6889)
# - Allow NTP (port 123)
```

LAB 3: Allow/Deny from Specific IPs

Objective: Control access from specific IP addresses

Difficulty: ★ Beginner

Time: 20 minutes

Prerequisites:

```
bash

# Lab 1-2 completed
sudo ufw status numbered
```

Lab Steps:

```
bash

# Step 1: Allow SSH only from specific IP
sudo ufw allow from 192.168.1.100 to any port 22

# Step 2: Allow HTTP from specific subnet
sudo ufw allow from 192.168.1.0/24 to any port 80

# Step 3: Deny traffic from untrusted IP
sudo ufw deny from 203.0.113.50

# Step 4: Allow from IP range to specific port
sudo ufw allow from 10.0.0.0/8 to any port 443

# Step 5: View all source-based rules
sudo ufw status numbered

# Step 6: More restrictive - deny all except specific IPs
# First, make sure you have a rule allowing your own IP
sudo ufw allow from 192.168.1.100 to any port 22
```

Validation:

```
bash

# From allowed IP (192.168.1.100), SSH should work:
ssh user@target-ip
# Should connect

# From other IP (if you have another machine), SSH should be blocked:
ssh user@target-ip
# Connection should timeout or be refused
```

Expected Output:

```
Status: active

      To                Action      From
      --                -
22/tcp                ALLOW      192.168.1.100
80/tcp                ALLOW      192.168.1.0/24
203.0.113.50          DENY      Anywhere
443/tcp               ALLOW      10.0.0.0/8
```

Common Mistakes:

- ❌ Not considering your own IP when restricting access
- ❌ Using wrong CIDR notation (use /24 for 255.255.255.0)
- ❌ Denying by IP then allowing all (rules match first match wins)

Post-Lab Questions:

1. What is CIDR notation and why 192.168.1.0/24?
2. In what order do UFW rules get evaluated?
3. Why use "deny from IP" instead of denying just the port?

Challenge Exercise:

```
bash

# Scenario: Allow SSH from headquarters (10.0.0.0/8)
# and home office (192.168.1.0/24), but deny from IP 192.168.1.50
# (rogue device on same network)
```

LAB 4: Rate Limiting and DDoS Protection

Objective: Implement rate limiting to prevent brute force and DDoS

Difficulty: ★★ Beginner+

Time: 20 minutes

Prerequisites:

```
bash

sudo ufw status
# UFW must be enabled
```

Lab Steps:

```
bash

# Step 1: Delete old SSH rule (if exists)
sudo ufw status numbered
sudo ufw delete 1 # Replace 1 with actual SSH rule number

# Step 2: Add rate-limited SSH
sudo ufw limit 22/tcp comment 'SSH rate limit'

# Step 3: Add rate-limited HTTP
sudo ufw limit 80/tcp comment 'HTTP rate limit'

# Step 4: Verify rate limit rules
sudo ufw status verbose

# Step 5: View rule details
sudo ufw status numbered

# Step 6: Test rate limiting on port 22
# Connect 6 times rapidly (will fail on 6th)
for i in {1..10}; do
    echo "Attempt $i:"
    timeout 1 ssh -o ConnectTimeout=1 user@localhost &
    sleep 0.1
done
wait
```

Validation:

```
bash
```

```
# Test SSH rate limiting
# Rapid connections should eventually fail:
for i in {1..20}; do
    nc -w1 localhost 22 > /dev/null 2>&1 && echo "Connection $i: SUCCESS" || ec
    sleep 0.1
done

# After ~5 successful connections, others should be blocked
```

Expected Output:

```
Status: active
      To                Action      From
      --                -
22/tcp                LIMIT      Anywhere
80/tcp                LIMIT      Anywhere
```

Rate Limiting Details:

```
UFW limit = 5 connections per minute after burst of 10
- First 10 connections: allowed instantly (burst)
- 11th+ connection: rate limited to 5/min
- After 1 minute of no connections: burst resets
```

Common Mistakes:

- ❌ Confusing "limit" with "allow"
- ❌ Expecting immediate blocking after limit
- ❌ Not understanding burst behavior

Post-Lab Questions:

1. What's the difference between LIMIT and ALLOW?
2. How many connections can be made before rate limiting kicks in?
3. When is rate limiting useful vs just denying?

Challenge Exercise:

```
bash
```



```
# Create scenario where:  
# - Port 22 (SSH): 3 connections per minute  
# - Port 80 (HTTP): unlimited from trusted subnet, limited from others  
# - Port 443 (HTTPS): limit to 10/minute
```

LAB 5: Removing and Managing UFW Rules

Objective: Master rule deletion and management

Difficulty: ★★ Beginner+

Time: 15 minutes

Prerequisites:

```
bash  
  
# Labs 1-4 completed with multiple rules created  
sudo ufw status numbered
```

Lab Steps:

```
bash
```

```
# Step 1: View all current rules with line numbers
sudo ufw status numbered

# Step 2: Delete rule by line number (example: delete rule #3)
sudo ufw delete 3

# Step 3: Verify rule deleted
sudo ufw status numbered

# Step 4: Delete rule by specification
sudo ufw delete allow 80/tcp

# Step 5: Delete deny rule
sudo ufw delete deny from 203.0.113.50

# Step 6: Insert rule at specific position (insert before rule #2)
sudo ufw insert 2 allow 3306/tcp comment 'MySQL'

# Step 7: Verify insertion worked
sudo ufw status numbered

# Step 8: Disable UFW (don't delete rules)
sudo ufw disable

# Step 9: Check status (should show inactive but rules saved)
sudo ufw status

# Step 10: Re-enable UFW (rules should reappear)
sudo ufw enable

# Step 11: Verify rules persist
sudo ufw status numbered
```

Validation:

```
bash

# After disable/enable cycle:
sudo ufw enable
sudo ufw status

# All previous rules should still be there
```

Important: Never use `ufw reset` unless you want to delete ALL rules:

```
bash
```

```
# DESTRUCTIVE - deletes all rules
sudo ufw reset
# Answer 'yes' to confirm
```

Common Mistakes:

- ❌ Using `ufw delete` with wrong rule number
- ❌ Accidentally running `ufw reset`
- ❌ Not tracking which rule number corresponds to which service

Best Practice:

```
bash
```

```
# Always view numbered list before deleting:
sudo ufw status numbered # THEN look at numbers
sudo ufw delete 5        # THEN delete by number
```

Post-Lab Questions:

1. What's the difference between `delete allow 80` and `delete 3`?
2. Why would you insert a rule instead of append?
3. What happens to rules when you disable UFW?

Challenge Exercise:

```
bash
```

```
# Scenario: You created 10 rules but only want to keep SSH and HTTPS
# Delete all others using line numbers
```

LEVEL 2: iptables BASICS (Labs 6-10)

LAB 6: iptables Installation and Default Policies

Objective: Install iptables and set secure default policies

Difficulty: ★★ Beginner+

Time: 15 minutes

Prerequisites:

```
bash

# Disable UFW first (to avoid conflicts)
sudo ufw disable
sudo ufw reset # When prompted, answer 'yes'

# Verify UFW is disabled
sudo ufw status
# Should show: inactive
```

Lab Steps:

```
bash
```

```
# Step 1: Install iptables and persistence package
sudo apt install -y iptables iptables-persistent netfilter-persistent

# Step 2: Check current policies (likely all ACCEPT)
sudo iptables -L | grep "Chain"
sudo iptables -L | grep "policy"

# Step 3: CRITICAL - Allow loopback interface
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT

# Step 4: Allow established and related connections
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Step 5: Allow SSH (BEFORE changing default policy!)
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Step 6: Set secure default policies
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

# Step 7: Verify policies set
sudo iptables -L | grep "policy"

# Step 8: View all rules
sudo iptables -L -n -v

# Step 9: Save rules for persistence
sudo iptables-save | sudo tee /etc/iptables/rules.v4

# Step 10: Verify rules persist on reboot
sudo systemctl enable netfilter-persistent
sudo systemctl status netfilter-persistent
```

Validation:

```
bash
```

```
# Verify SSH still works
ssh localhost
# Should connect successfully

# Verify loopback works
ping 127.0.0.1
# Should respond

# View packet counts (should see traffic in established/loopback rules)
sudo iptables -L -n -v
```

Expected Output:

```
Chain INPUT (policy DROP 0 packets, 0 bytes)
Chain FORWARD (policy DROP 0 packets, 0 bytes)
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)

Rules should show:
- Loopback accept
- Established/related accept
- SSH accept
```

Common Mistakes:

- ❌ Setting DROP policy before allowing loopback (system hangs)
- ❌ Setting DROP policy before allowing established (new SSH fails)
- ❌ Not saving rules (lost on reboot)
- ❌ Confusing -P (policy) with -A (append)

Emergency Recovery:

```
bash

# If you get locked out, reboot from VM console:
sudo reboot

# Or reset all policies to ACCEPT (temporary):
sudo iptables -P INPUT ACCEPT
sudo iptables -P OUTPUT ACCEPT
sudo iptables -P FORWARD ACCEPT
sudo iptables -F # Flush all rules
```

Post-Lab Questions:

1. Why allow loopback before setting DROP policy?
 2. What's ESTABLISHED,RELATED state used for?
 3. Why is persistence important?
-

LAB 7: Adding Rules to iptables Chains

Objective: Add specific allow/deny rules to iptables chains

Difficulty: ★★ Intermediate

Time: 20 minutes

Prerequisites:

```
bash

# Lab 6 completed with secure policies
sudo iptables -L | grep "policy"
# Should show: DROP for INPUT/FORWARD, ACCEPT for OUTPUT
```

Lab Steps:

```
bash
```

```
# Step 1: View current rules with line numbers
sudo iptables -L INPUT -n --line-numbers

# Step 2: Allow HTTP (port 80)
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# Step 3: Allow HTTPS (port 443)
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Step 4: Allow DNS (both TCP and UDP on port 53)
sudo iptables -A INPUT -p udp --dport 53 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 53 -j ACCEPT

# Step 5: Allow ICMP (ping)
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT

# Step 6: Verify all rules added
sudo iptables -L INPUT -n -v

# Step 7: List rules in command format (easy to understand)
sudo iptables -L INPUT -n --line-numbers

# Step 8: Test HTTP (from another machine or locally)
curl http://localhost
# May fail if no web server, but firewall won't block

# Step 9: Test DNS
nslookup google.com
# Should work if DNS server accessible

# Step 10: Test ICMP
ping 127.0.0.1
# Should respond
```

Validation:

```
bash
```



```
# SSH should work (from Lab 6)
ssh localhost
# ✓ Connected

# HTTP should work
timeout 2 bash -c 'echo > /dev/tcp/127.0.0.1/80'
# Exit code 0 = connection possible

# DNS should work
dig google.com @8.8.8.8
# Should get response

# Ping should work
ping -c 2 127.0.0.1
# Should respond

# Blocked ports should fail (e.g., Telnet on 23)
timeout 2 nc localhost 23
# Should timeout/fail
```

Expected Output:

```
Chain INPUT (policy DROP)
target      prot opt source                destination
ACCEPT      all  --  anywhere               anywhere
ACCEPT      all  --  anywhere               anywhere
ACCEPT      tcp  --  anywhere               anywhere    tcp dpt:22
ACCEPT      tcp  --  anywhere               anywhere    tcp dpt:80
ACCEPT      tcp  --  anywhere               anywhere    tcp dpt:443
ACCEPT      udp  --  anywhere               anywhere    udp dpt:domain
ACCEPT      tcp  --  anywhere               anywhere    tcp dpt:domain
ACCEPT      icmp --  anywhere               anywhere    icmptype 8
```

Common Mistakes:

- ✗ Forgetting to specify protocol (-p tcp or -p udp)
- ✗ Using wrong port numbers
- ✗ Not specifying --dport (might match source port instead)
- ✗ Mixing up -A (append) with -I (insert)

Rule Order Matters:

```
bash
```

```
# First matching rule wins!
# This denies 192.168.1.100 access:
sudo iptables -A INPUT -s 192.168.1.100 -j DROP
sudo iptables -A INPUT -s 192.168.1.100 -p tcp --dport 22 -j ACCEPT
# WRONG ORDER: The ACCEPT rule never reaches because DROP matched first!

# Correct order:
sudo iptables -A INPUT -s 192.168.1.100 -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -s 192.168.1.100 -j DROP
# CORRECT: ACCEPT rule checked first
```

Post-Lab Questions:

1. Why must we specify protocol (-p) for port-based rules?
2. What happens if we don't specify ICMP type?
3. How do you add a rule in a specific position?

Challenge Exercise:

```
bash
```

```
# Add these rules in correct order:
# - Allow MySQL from 192.168.1.100 only
# - Allow NTP (port 123) from anywhere
# - Allow Syslog (port 514 UDP) from 10.0.0.0/8
# - Block all other traffic
```

LAB 8: Deleting and Modifying iptables Rules

Objective: Master rule deletion and modification

Difficulty: ★★ Intermediate

Time: 20 minutes

Prerequisites:

```
bash
```

```
# Lab 7 completed with multiple rules
sudo iptables -L INPUT -n --line-numbers
```

Lab Steps:

bash

Step 1: View current rules with line numbers

```
sudo iptables -L INPUT -n --line-numbers
```

Step 2: Delete rule by line number (example: delete rule #5)

```
sudo iptables -D INPUT 5
```

Step 3: Verify deletion

```
sudo iptables -L INPUT -n --line-numbers
```

Step 4: Delete rule by full specification

```
sudo iptables -D INPUT -p tcp --dport 80 -j ACCEPT
```

Step 5: Verify deletion

```
sudo iptables -L INPUT -n --line-numbers
```

Step 6: Insert rule at specific position (before line 3)

```
sudo iptables -I INPUT 3 -p tcp --dport 3306 -j ACCEPT
```

Step 7: Verify insertion

```
sudo iptables -L INPUT -n --line-numbers
```

Step 8: Replace a rule at position

```
sudo iptables -R INPUT 3 -p tcp --dport 3306 -s 192.168.1.0/24 -j ACCEPT
```

Step 9: Verify replacement

```
sudo iptables -L INPUT -n --line-numbers
```

Step 10: Flush specific chain (delete all rules in INPUT)

```
sudo iptables -F INPUT
```

Step 11: Verify chain is empty

```
sudo iptables -L INPUT
```

Step 12: Restore from backup (if you saved earlier)

```
sudo iptables-restore < /etc/iptables/rules.v4
```

Step 13: Verify restored

```
sudo iptables -L INPUT -n -v
```

Validation:

Difference Between Operations:

bash

-D = Delete (exact match required)
-I = Insert at position (shifts existing rules down)
-R = Replace at position (overwrites existing rule)
-F = Flush (delete all rules **in** chain)

Common Mistakes:

- **✗** Deleting by line number that shifts after first delete
- **✗** Forgetting to specify all parameters for -D
- **✗** Using -F instead of -D (deletes too much!)

Safe Deletion Strategy:

bash

```
# 1. View with line numbers
sudo iptables -L INPUT -n --line-numbers

# 2. View specific rule to confirm
sudo iptables -L INPUT -n | grep "port-number"

# 3. Delete ONLY when you're sure
sudo iptables -D INPUT [line-number]

# 4. Verify immediately
sudo iptables -L INPUT -n --line-numbers
```

Post-Lab Questions:

1. What's the difference between -D and -R?
2. Why should you view line numbers before deleting?
3. How would you safely modify a rule?

Challenge Exercise:

```
bash

# Scenario: You have 15 rules and want to:
# 1. Move SSH rule to position #1
# 2. Delete the old 80/tcp rule
# 3. Add MySQL with source restriction
# Do this without losing SSH access!
```

LAB 9: Rate Limiting and Advanced Matching

Objective: Implement rate limiting and complex packet matching in iptables

Difficulty: ★★☆☆ Intermediate

Time: 25 minutes

Prerequisites:

```
bash

# Lab 8 completed
# Current iptables rules should allow basic traffic
```

Lab Steps:

bash

Step 1: View current rules

```
sudo iptables -L INPUT -n --line-numbers
```

Step 2: Add rate-limiting rule for SSH (5 connections per minute)

```
sudo iptables -A INPUT -p tcp --dport 22 -m limit --limit 5/minute --limit-burst 10
```

Step 3: Drop any SSH after rate limit exceeded

```
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

Step 4: Add rate-limiting for HTTP (25 connections per hour)

```
sudo iptables -A INPUT -p tcp --dport 80 -m limit --limit 25/hour --limit-burst 100
```

```
sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

Step 5: Match multiple ports at once

```
sudo iptables -A INPUT -p tcp -m multiport --dports 443,465,587,993 -j ACCEPT
```

Step 6: Match port range (allow dynamic ephemeral ports)

```
sudo iptables -A INPUT -p tcp --dport 8000:9000 -j ACCEPT
```

Step 7: Match specific source IP + port combination

```
sudo iptables -A INPUT -s 192.168.1.100 -p tcp --dport 3306 -j ACCEPT
```

Step 8: Match source range with destination port

```
sudo iptables -A INPUT -s 10.0.0.0/8 -p tcp --dport 443 -j ACCEPT
```

Step 9: Match by interface

```
sudo iptables -A INPUT -i eth0 -p tcp --dport 22 -j ACCEPT
```

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

Step 10: Match by destination IP (if multiple IPs on server)

```
sudo iptables -A INPUT -d 192.168.1.50 -p tcp --dport 80 -j ACCEPT
```

Step 11: View all rules with conditions

```
sudo iptables -L INPUT -n -v
```

Step 12: Save configuration

```
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

Testing Rate Limiting:

```
bash
```

```
# Test SSH rate limiting
```

```
# Rapid sequential connections to see when limit kicks in
```

```
for i in {1..15}; do
```

```
    (timeout 1 ssh -o ConnectTimeout=1 user@localhost >/dev/null 2>&1) && echo
```

```
    sleep 0.1
```

```
done
```

```
# Results:
```

```
# First 10 attempts: Connected (burst)
```

```
# Attempt 11-15: Blocked (rate limited)
```

Validation:

```
bash
```

```
# Check rate limiting is active
```

```
sudo iptables -L INPUT -n -v | grep "limit"
```

```
# Verify multiport rule
```

```
sudo iptables -L INPUT -n | grep "multiport"
```

```
# Verify port range
```

```
sudo iptables -L INPUT -n | grep "8000:9000"
```

Rate Limiting Concepts:

```
--limit 5/minute = 5 packets per minute
```

```
--limit-burst 10 = Allow up to 10 packets instantly, then apply limit
```

```
--limit 25/hour = 25 packets per hour
```

```
--limit 1/second = 1 packet per second
```

```
Common defaults:
```

```
SSH: limit 5/minute --limit-burst 10
```

```
HTTP: limit 25/hour --limit-burst 100
```

```
DNS: limit 10/second --limit-burst 20
```

Advanced Matching Module:

```
bash
```

```
# -m = use matching module
# Common modules:
# - limit: rate limiting
# - multiport: multiple ports
# - state: connection state
# - conntrack: connection tracking
# - comment: add labels to rules
```

Post-Lab Questions:

1. What's the difference between --limit and --limit-burst?
2. Why have both ACCEPT with limit and DROP after?
3. How do multiport rules simplify firewall configuration?

Challenge Exercise:

```
bash

# Create this scenario:
# - Allow SSH: 5 connections/min from anywhere
# - Allow HTTP/HTTPS: unlimited from 192.168.1.0/24, limited from others
# - Allow MySQL: only from 10.0.0.0/8
# - Block everything else
```

LAB 10: Logging and Monitoring iptables

Objective: Configure logging to monitor traffic and debug firewall

Difficulty: ★★★ Intermediate

Time: 20 minutes

Prerequisites:

```
bash

# Lab 9 completed with rate limiting rules
```

Lab Steps:

```
bash
```



```
# Step 1: Add logging rule BEFORE drop rule (first match wins!)
# This logs all dropped SSH connections
sudo iptables -I INPUT -p tcp --dport 22 -m limit --limit 5/minute --limit-burst 5 -j LOG --log-prefix "[SSH_DROPPED] " -j DROP

# Step 2: Add logging for general dropped packets
sudo iptables -A INPUT -m limit --limit 5/min -j LOG --log-prefix "[UFW-ALLOW-INPUT-DROPPED] " -j DROP
sudo iptables -A INPUT -j DROP

# Step 3: Verify logging rules added
sudo iptables -L INPUT -n -v

# Step 4: View kernel logs where iptables logs to
sudo tail -f /var/log/kern.log

# Step 5: In another terminal, trigger some dropped connections
# Try connecting to a blocked port:
timeout 1 nc localhost 23
# Watch the kernel log for entries

# Step 6: Check how many packets matched each rule
sudo iptables -L INPUT -n -v
# Look at the "pkts" column

# Step 7: Use dmesg for real-time kernel messages
sudo dmesg | tail -20

# Step 8: Check syslog
sudo tail -f /var/log/syslog | grep "SSH_DROPPED"

# Step 9: Create custom log file for iptables
sudo nano /etc/rsyslog.d/30-iptables.conf

# Add this content:
# :msg, contains, "UFW" /var/log/iptables.log
# & stop

# Step 10: Restart rsyslog
sudo systemctl restart rsyslog

# Step 11: Verify separate log file
sudo tail -f /var/log/iptables.log
```

```
# Step 12: Analyze logs with grep
sudo grep "SSH_DROPPED" /var/log/kern.log
sudo grep "DROPPED" /var/log/kern.log | wc -l # Count dropped packets
```

Log Entry Format:

```
[SSH_DROPPED] IN=eth0 OUT= MAC=... SRC=192.168.1.100 DST=10.0.0.1 PROTO=TCP
SPT=12345 DPT=22 WINDOW=65535 SYN URGP=0
```

Decoding Log Entries:

```
IN=eth0           = Incoming interface
SRC=192.168.1.100 = Source IP
DST=10.0.0.1      = Destination IP
PROTO=TCP         = Protocol
SPT=12345         = Source Port
DPT=22           = Destination Port
SYN              = TCP flag (new connection)
```

Validation:

```
bash

# Verify logging is working:
sudo iptables -L INPUT -n -v | head -10

# Should show non-zero packet counts for logging rules

# View actual log entries:
sudo tail -f /var/log/kern.log | grep "SSH"

# In another terminal:
timeout 1 nc localhost 22

# Back in first terminal, you should see log entries
```

Common Mistakes:

- ❌ Adding LOG rule AFTER DROP (never logs!)
- ❌ Using too high log-prefix causing log flooding
- ❌ Not using --log-level correctly
- ❌ Not limiting log output (can cause disk full)

Log Levels:

```
0 = emerg
1 = alert
2 = crit
3 = err
4 = warning (default)
5 = notice
6 = info
7 = debug
```

Post-Lab Questions:

1. Why must LOG rule come BEFORE DROP rule?
2. How would you prevent log flooding?
3. What information do you get from iptables logs?

Challenge Exercise:

```
bash

# Create logging that:
# - Logs all dropped connections with prefix "[BLOCKED]"
# - Logs SSH attempts separately with "[SSH-ATTEMPT]"
# - Sends logs to /var/log/firewall.log
# - Limits log rate to 5 entries per minute
```

LEVEL 3: ADVANCED TOPICS (Labs 11-15)

LAB 11: UFW + iptables Coexistence

Objective: Run both UFW and iptables together safely

Difficulty: ★★☆☆ Advanced

Time: 25 minutes

Prerequisites:

```
bash
```

```
# Current iptables rules working
sudo iptables -L INPUT -n --line-numbers | head -20

# UFW is disabled
sudo ufw status
```

Lab Steps:

```
bash
```

```
# Step 1: Backup current iptables rules
sudo iptables-save > /tmp/iptables-backup-before-ufw.rules

# Step 2: Enable UFW (will not delete iptables rules, but adds its own)
sudo ufw enable

# Step 3: View both rule sets
echo "=== UFW Rules ==="
sudo ufw status numbered
echo "=== iptables Rules ==="
sudo iptables -L INPUT -n | head -20

# Step 4: Both are active! UFW's rules wrap iptables
sudo iptables -L | grep "Chain"

# Step 5: Add UFW rule
sudo ufw allow 8080/tcp

# Step 6: View that UFW rule was added to iptables
sudo iptables -L INPUT -n | grep 8080

# Step 7: Add iptables rule directly
sudo iptables -A INPUT -p tcp --dport 9000 -j ACCEPT

# Step 8: View both UFW and iptables rules
sudo iptables -L INPUT -n -v

# Step 9: Check UFW doesn't show iptables rule
sudo ufw status numbered
# UFW won't show the direct iptables rule (9000)

# Step 10: Disable UFW and verify iptables rules persist
sudo ufw disable

# Step 11: Check iptables rules still there
sudo iptables -L INPUT -n -v | grep 9000
# Should still see the 9000 rule

# Step 12: Re-enable UFW
sudo ufw enable

# Step 13: Verify both rule sets exist
sudo iptables -L INPUT -n -v | grep -E "8080|9000"
```

```
# Step 14: Save final state
sudo iptables-save | sudo tee /etc/iptables/rules.v4
sudo ufw status > /tmp/ufw-rules.txt
```

Understanding the Interaction:

When UFW is enabled:
UFW creates iptables rules that wrap around raw iptables
UFW rules: Entry point for packet filtering
Custom iptables rules: Applied after UFW rules

Order of processing:

1. UFW's main rules (user rules)
2. UFW's default policies
3. Custom iptables rules (appended after UFW)
4. Kernel decision (DROP/ACCEPT)

Validation:

```
bash

# Test port 8080 (UFW rule) works
timeout 1 bash -c 'echo > /dev/tcp/127.0.0.1/8080'
# Exit code 0 = working

# Test port 9000 (iptables rule) works
timeout 1 bash -c 'echo > /dev/tcp/127.0.0.1/9000'
# Exit code 0 = working

# Test blocked port fails
timeout 1 bash -c 'echo > /dev/tcp/127.0.0.1/23'
# Exit code non-0 = blocked
```

Common Pitfalls:

- ❌ Thinking UFW replaces iptables (it doesn't, it wraps it)
- ❌ Not realizing custom iptables rules are invisible to UFW
- ❌ Deleting UFW rules expecting to affect iptables rules
- ❌ Confusion about rule evaluation order

When to Use Both:

Use UFW for:

- ✓ Regular allow/deny rules
- ✓ Simple management interface
- ✓ Per-IP access control

Use iptables for:

- ✓ Advanced matching (multiport, port ranges, etc.)
- ✓ Rate limiting
- ✓ Logging and debugging
- ✓ NAT/Port forwarding
- ✓ Rules UFW can't express

Post-Lab Questions:

1. Does UFW replace or wrap iptables?
2. Which rules get evaluated first - UFW or custom iptables?
3. Why might you want both?

Challenge Exercise:

```
bash

# Setup scenario:
# - UFW: Allow HTTP, HTTPS, SSH to specific subnet
# - iptables: Add rate limiting to SSH
# - Both: Work together without conflicts
```

LAB 12: NAT and Port Forwarding

Objective: Configure Network Address Translation and port forwarding

Difficulty: ★★★★★ Advanced

Time: 30 minutes

Prerequisites:

```
bash
```

```
# UFW disabled (UFW doesn't support NAT)
sudo ufw disable

# iptables rules in place
sudo iptables -L FORWARD -n
```

Lab Setup (requires 2 machines/VMs):

```
Machine A (Firewall/Router): 192.168.1.10
└─ Can reach 192.168.2.20 (internal network)
└─ Has public IP 203.0.113.1

Machine B (Internal): 192.168.2.20
└─ Private IP only
└─ Needs to be reachable from outside via Machine A
```

Lab Steps:

```
bash
```



```
# Step 1: Enable IP forwarding (critical for routing)
sudo sysctl -w net.ipv4.ip_forward=1

# Persist setting across reboots:
echo "net.ipv4.ip_forward = 1" | sudo tee /etc/sysctl.conf

# Step 2: Allow forwarding in firewall
sudo iptables -A FORWARD -i eth0 -o eth1 -j ACCEPT
sudo iptables -A FORWARD -i eth1 -o eth0 -m state --state ESTABLISHED,RELATED -j ACCEPT

# Step 3: Setup Port Forwarding (forward 8080 to internal machine port 80)
# External machine connects to 203.0.113.1:8080
# Gets forwarded to 192.168.2.20:80

# Enable PREROUTING to modify destination
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 8080 -j DNAT --to-destination 192.168.2.20:80

# Enable POSTROUTING for response packets to work
sudo iptables -t nat -A POSTROUTING -o eth1 -p tcp --dport 80 -d 192.168.2.20 -j MASQUERADE

# Step 4: View NAT rules (different table than filter!)
sudo iptables -t nat -L -n -v

# Step 5: Test port forwarding (from external machine)
curl http://203.0.113.1:8080
# Should reach web server on 192.168.2.20:80

# Step 6: SSH port forwarding example
# Forward port 2222 to SSH on internal machine
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 2222 -j DNAT --to-destination 192.168.2.20:22
sudo iptables -t nat -A POSTROUTING -o eth1 -p tcp --dport 22 -d 192.168.2.20 -j MASQUERADE

# Test SSH forwarding
ssh -p 2222 user@203.0.113.1
# Should reach SSH on 192.168.2.20

# Step 7: MASQUERADING (simpler than SNAT for dynamic IPs)
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# Step 8: View all NAT rules including MASQUERADE
sudo iptables -t nat -L -n -v

# Step 9: Delete specific port forward
```

```
sudo iptables -t nat -D PREROUTING -i eth0 -p tcp --dport 8080 -j DNAT --to-des  
  
# Step 10: Flush all NAT rules (be careful!)  
sudo iptables -t nat -F  
  
# Step 11: Save configuration  
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

Understanding NAT Concept:

Normal packet flow:
Client → Server
(Client sees direct connection)

With NAT/Port Forwarding:
Client → Router:8080 → PREROUTING NAT → Server:80
Server → Router → POSTROUTING MASQUERADE → Client
(Client sees Router, not real server)

DNAT vs SNAT:

DNAT (Destination NAT): Modify destination IP/port

- Used for port forwarding/load balancing
- External client → router → internal server

SNAT (Source NAT): Modify source IP/port

- Used for NAT outbound traffic
- Internal client → router → external server

MASQUERADE: Dynamic SNAT

- Like SNAT but works with dynamic IPs
- Useful for routers with changing public IPs

Validation:

```
bash
```

```
# From external machine:
curl http://203.0.113.1:8080
# Should work if port forwarding correct

# Check NAT translation happening:
sudo tcpdump -i eth0 -n port 8080
# Shows traffic on port 8080 going to 192.168.2.20

# View NAT connections
sudo conntrack -L | grep "192.168.2.20"
# Shows active NAT translations
```

Common Mistakes:

- ❌ Forgetting to enable IP forwarding
- ❌ Not setting POSTROUTING (response packets get dropped)
- ❌ Wrong interface names (-i eth0 vs -o eth1)
- ❌ Confusing source and destination port numbers

Post-Lab Questions:

1. What's the difference between DNAT and SNAT?
2. Why is IP forwarding necessary?
3. When would you use MASQUERADE instead of SNAT?

Challenge Exercise:

```
bash

# Setup complex forwarding:
# - Port 80 (HTTP) → 192.168.2.20:8080
# - Port 443 (HTTPS) → 192.168.2.20:8443
# - Port 3306 (MySQL) → 192.168.2.30:3306
# - All from different internal IPs
```

LAB 13: Connection Tracking and Stateful Filtering

Objective: Understand and implement stateful packet filtering

Difficulty: ★★★★★ Advanced

Time: 25 minutes

Prerequisites:

```
bash
```

```
# UFW disabled
```

```
sudo ufw disable
```

```
# Basic iptables rules in place
```

```
sudo iptables -L -n
```

Lab Steps:

```
bash
```

```
# Step 1: Check if conntrack module loaded
lsmod | grep conntrack

# If not loaded, load it:
sudo modprobe nf_conntrack

# Step 2: View current connection tracking state
sudo cat /proc/net/nf_conntrack

# Step 3: Install conntrack-tools for better viewing
sudo apt install -y conntrack

# Step 4: View active connections (better format)
sudo conntrack -L

# Step 5: Create stateful rules
sudo iptables -F INPUT # Clear existing input rules

# Step 6: Allow loopback
sudo iptables -A INPUT -i lo -j ACCEPT

# Step 7: Allow established and related connections (key rule!)
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Alternative newer syntax:
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# Step 8: Allow new SSH connections
sudo iptables -A INPUT -p tcp --dport 22 -m state --state NEW -j ACCEPT

# Step 9: Drop invalid connections
sudo iptables -A INPUT -m state --state INVALID -j DROP

# Step 10: Set default policy
sudo iptables -P INPUT DROP

# Step 11: Test SSH connection (should work because of ESTABLISHED rule)
# SSH OUT = NEW connection
# SSH responses = ESTABLISHED connection (allowed)
ssh user@localhost
# Should work!

# Step 12: View conntrack entries
```

```

sudo conntrack -L | grep ssh

# Step 13: Make HTTP outbound connection
curl http://google.com
# Should work (NEW out, ESTABLISHED back)

# Step 14: View that connection in conntrack
sudo conntrack -L | grep tcp.*http

# Step 15: View connection states
sudo conntrack -L | head -5

# Step 16: Connection state values:
# - NEW: new connection
# - ESTABLISHED: existing connection
# - RELATED: related to existing (e.g., FTP data channel)
# - INVALID: malformed or wrong state
# - UNTRACKED: exempt from tracking

# Step 17: Untracked example (marked to bypass tracking)
sudo iptables -A INPUT -j ACCEPT # This bypasses state tracking

# Step 18: Show connection tracking stats
sudo cat /proc/sys/net/netfilter/nf_conntrack_max
# Maximum tracked connections

# Step 19: View current connection count
sudo conntrack -L | wc -l

# Step 20: Save rules
sudo iptables-save | sudo tee /etc/iptables/rules.v4

```

Connection States:

```

NEW           = First packet of connection
ESTABLISHED   = Follow-up packets, both directions seen
RELATED       = Related to existing connection (FTP data, ICMP error, etc.)
INVALID       = Can't identify state (malformed, unsynchronized)
UNTRACKED     = Not tracked by conntrack module

```

Stateful vs Stateless:

```
STATELESS (old way):
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT      # Inbound
sudo iptables -A INPUT -p tcp --sport 22 -j ACCEPT      # Inbound responses
# Must allow both directions explicitly!

STATEFUL (modern way):
sudo iptables -A INPUT -m state --state NEW -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED -j ACCEPT
# Only need to define NEW direction, responses automatic!
```

Validation:

```
bash

# Test stateful filtering
curl https://example.com
# Should work (outbound NEW, inbound ESTABLISHED)

# Check connection in tracking table
sudo conntrack -L | grep example.com

# Verify states
sudo conntrack -L | grep -E "ESTABLISHED|NEW|RELATED"
```

Common Mistakes:

-  Forgetting ESTABLISHED rule (connections hang)
-  Confusing NEW with ESTABLISHED
-  Not understanding RELATED (FTP, DNS issues)
-  Blocking INVALID without understanding implications

Real-World Impact:

```
bash
```

```
# Without stateful rules:
# Outbound SSH fails (responses dropped)
# curl fails (responses dropped)
# Most connections timeout

# With stateful rules:
# Everything works naturally
# Saves massive rulebase
```

Post-Lab Questions:

1. Why is ESTABLISHED rule critical?
2. What's the difference between NEW and RELATED?
3. How does conntrack help firewall security?

Challenge Exercise:

```
bash

# Create complete stateful ruleset:
# - Allow SSH (NEW)
# - Allow HTTP/HTTPS (NEW)
# - Allow outbound connections
# - Block everything else
# Test that responses work through ESTABLISHED tracking
```

LAB 14: Blocking and Preventing Common Attacks

Objective: Configure firewall to block common attack patterns

Difficulty: ★★★★★ Advanced

Time: 30 minutes

Prerequisites:

```
bash

# Lab 13 completed with stateful filtering
sudo iptables -L INPUT -n | grep ESTABLISHED
```

Lab Steps:

bash

Step 1: View current rules

```
sudo iptables -L INPUT -n --line-numbers
```

Step 2: BLOCK SYN FLOOD ATTACK

SYN flood: attacker sends massive SYN packets to consume resources

```
sudo iptables -A INPUT -p tcp --syn -m limit --limit 5/s --limit-burst 10 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --syn -j DROP
```

Step 3: BLOCK PORT SCANNING

Block UDP packets to random ports (indicator of port scan)

```
sudo iptables -N port-scanning
```

```
sudo iptables -A port-scanning -p tcp --tcp-flags SYN,ACK,FIN,RST RST -m limit
```

```
sudo iptables -A port-scanning -j DROP
```

Step 4: BLOCK INVALID PACKETS

Malformed packets or wrong state transitions

```
sudo iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
```

```
sudo iptables -A INPUT -p tcp ! --syn -m state --state NEW -j DROP
```

Step 5: BLOCK COMMON ATTACKS

FIN scan (all flags set)

```
sudo iptables -A INPUT -p tcp --tcp-flags ALL FIN -j DROP
```

Xmas scan (FIN, PSH, URG)

```
sudo iptables -A INPUT -p tcp --tcp-flags ALL FIN,PSH,URG -j DROP
```

Null scan (no flags)

```
sudo iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
```

Step 6: PREVENT IP SPOOFING

Drop if source is RFC 1918 private IP from WAN interface

```
sudo iptables -A INPUT -i eth0 -s 10.0.0.0/8 -j DROP
```

```
sudo iptables -A INPUT -i eth0 -s 192.168.0.0/16 -j DROP
```

```
sudo iptables -A INPUT -i eth0 -s 172.16.0.0/12 -j DROP
```

Drop if source is 0.0.0.0 (invalid)

```
sudo iptables -A INPUT -s 0.0.0.0/8 -j DROP
```

Step 7: PREVENT ICMP ATTACKS

Allow ICMP echo request (ping) but rate limit

```
sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 5/s --
```

```
sudo iptables -A INPUT -p icmp -j DROP
```

```

# Step 8: PREVENT LAND ATTACK
# Drop if source IP equals destination IP
sudo iptables -A INPUT -s 127.0.0.0/8 ! -i lo -j DROP

# Step 9: BLOCK FRAGMENTED PACKETS
# Fragmentation can hide exploits
sudo iptables -A INPUT -f -j DROP

# Step 10: SSH BRUTE FORCE PROTECTION
# Only 5 new SSH attempts per minute per IP
sudo iptables -A INPUT -p tcp --dport 22 -m limit --limit 5/m --limit-burst 5 -j DROP
sudo iptables -A INPUT -p tcp --dport 22 -j DROP

# Step 11: DNS AMPLIFICATION ATTACK PREVENTION
# Limit DNS queries
sudo iptables -A INPUT -p udp --dport 53 -m limit --limit 5/s --limit-burst 10 -j DROP
sudo iptables -A INPUT -p udp --dport 53 -j DROP

# Step 12: DDOS PROTECTION - CONNECTION LIMIT
# Max 100 connections per IP
sudo iptables -A INPUT -p tcp --dport 80 -m limit --limit 25/minute --limit-burst 100 -j DROP
sudo iptables -A INPUT -p tcp --dport 80 -j DROP

# Step 13: LOG SUSPICIOUS ACTIVITY BEFORE DROP
# Add logging before DROP rules:
sudo iptables -I INPUT -p tcp --tcp-flags ALL FIN -j LOG --log-prefix "[FIN_SCAN]"

# Step 14: View all attack prevention rules
sudo iptables -L INPUT -n -v

# Step 15: List rules with comments (if added)
sudo iptables -L -n -v | grep -E "syn|syn.*DROP|tcp-flags"

# Step 16: Save configuration
sudo iptables-save | sudo tee /etc/iptables/rules.v4

```

Attack Prevention Summary:

Attack Type	Prevention Rule
SYN Flood	Limit --syn traffic, rate limit
Port Scan	Drop resets/TCP flags, logging
Invalid Packets	Drop ALL flags NONE, invalid state
IP Spoofing	Drop RFC 1918 on public interface
ICMP Flood	Rate limit ICMP, drop excess
Land Attack	Drop source=destination
Fragmentation	Drop fragmented packets (-f)
SSH Brute Force	Limit new connections per IP
DNS Amplification	Rate limit DNS queries
DDoS	Connection limits, rate limiting

Validation:

```
bash

# Test FIN scan detection
sudo nmap -f -sF localhost
# Should be logged/dropped

# Test SYN flood prevention
sudo hping3 -S --flood localhost -p 22
# Should rate limit after burst

# View attack logs
sudo tail -f /var/log/kern.log | grep -E "FIN|SYN|SCAN"

# In another terminal, trigger attacks:
sudo nmap -sX localhost # Xmas scan
# Watch logs for blocks
```

Common Mistakes:

-  Being too aggressive (blocking legitimate traffic)
-  Not logging before drop (can't debug)
-  Forgetting to rate limit (blocks too quickly)
-  Not understanding TCP flags (confusing rule logic)

TCP Flags Explained:

```
SYN = Synchronize (new connection start)
ACK = Acknowledgement (response)
FIN = Finish (connection close)
RST = Reset (abort)
PSH = Push (immediate data)
URG = Urgent (urgent data)
```

```
Normal flow:  SYN → SYN-ACK → ACK → ...data... → FIN → FIN-ACK
Abnormal:     FIN-PSH-URG (Xmas)
              All flags none (Null)
              Only RST (Reset scan)
```

Post-Lab Questions:

1. Why rate limit SYN instead of blocking?
 2. What's the purpose of dropping fragmented packets?
 3. How would you detect and log an attack in real-time?
-

LAB 15: Complete Security Policy Configuration

Objective: Build complete, production-ready firewall security policy

Difficulty: ★★★★★ Advanced

Time: 45 minutes

Prerequisites:

```
bash

# All previous labs completed
# Understanding of stateful filtering, NAT, attack prevention
```

Lab Scenario:

```
Web Server Configuration:
- External IP: 203.0.113.1 (eth0)
- Internal IP: 192.168.1.10 (eth1)
- Services: HTTP (80), HTTPS (443), SSH (22)
- Backend servers on: 192.168.1.0/24
```

Lab Steps:

bash

```
# PART 1: BACKUP AND RESET
```

```
# Step 1: Backup current configuration
```

```
sudo iptables-save > /tmp/iptables-backup-lab15.rules  
echo "Backup saved to /tmp/iptables-backup-lab15.rules"
```

```
# Step 2: Reset to clean state
```

```
sudo iptables -F  
sudo iptables -F -t nat  
sudo iptables -F -t mangle  
sudo iptables -X  
sudo iptables -X -t nat  
sudo iptables -X -t mangle  
echo "Firewall reset to clean state"
```

```
# PART 2: SET SECURITY POLICIES
```

```
# Step 3: Set default policies (most important!)
```

```
sudo iptables -P INPUT DROP  
sudo iptables -P OUTPUT ACCEPT  
sudo iptables -P FORWARD DROP  
  
echo "Default policies set: INPUT=DROP, OUTPUT=ACCEPT, FORWARD=DROP"
```

```
# PART 3: ALLOW ESSENTIAL SERVICES
```

```
# Step 4: Allow loopback (required for system stability)
```

```
sudo iptables -A INPUT -i lo -j ACCEPT  
sudo iptables -A OUTPUT -o lo -j ACCEPT  
  
echo "Loopback allowed"
```

```
# Step 5: Allow established and related connections (stateful filtering)
```

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT  
sudo iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT  
  
echo "Stateful filtering enabled"
```

```
# Step 6: Allow SSH with rate limiting and source restrictions
```

```
# SSH from admin network only
```

```
sudo iptables -A INPUT -s 10.0.0.0/8 -p tcp --dport 22 -m state --state NEW -m  
sudo iptables -A INPUT -p tcp --dport 22 -j DROP  
  
echo "SSH configured (admin network only, rate limited)"
```

```
# PART 4: WEB SERVICES
```

```

# Step 7: Allow HTTP (port 80) - unlimited from anywhere
sudo iptables -A INPUT -p tcp --dport 80 -m state --state NEW -j ACCEPT

# Step 8: Allow HTTPS (port 443) - unlimited from anywhere
sudo iptables -A INPUT -p tcp --dport 443 -m state --state NEW -j ACCEPT

echo "HTTP/HTTPS allowed"

# PART 5: INTERNAL NETWORK COMMUNICATION
# Step 9: Allow internal network access
# Database and backend services only to internal network
sudo iptables -A INPUT -s 192.168.1.0/24 -p tcp --dport 3306 -j ACCEPT # MySQL
sudo iptables -A INPUT -s 192.168.1.0/24 -p tcp --dport 5432 -j ACCEPT # Postg
sudo iptables -A INPUT -s 192.168.1.0/24 -p tcp --dport 6379 -j ACCEPT # Redis

echo "Internal network services allowed"

# PART 6: ATTACK PREVENTION
# Step 10: Prevent SYN flood
sudo iptables -A INPUT -p tcp --syn -m limit --limit 5/s --limit-burst 10 -j AC
sudo iptables -A INPUT -p tcp --syn -j DROP

# Step 11: Prevent invalid packets
sudo iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP # Null scan
sudo iptables -A INPUT -p tcp --tcp-flags ALL FIN -j DROP # FIN scan
sudo iptables -A INPUT -p tcp ! --syn -m conntrack --ctstate NEW -j DROP # TCP

# Step 12: Prevent IP spoofing
sudo iptables -A INPUT -i eth0 -s 10.0.0.0/8 -j DROP # Private IPs on pu
sudo iptables -A INPUT -i eth0 -s 192.168.0.0/16 -j DROP
sudo iptables -A INPUT -i eth0 -s 172.16.0.0/12 -j DROP
sudo iptables -A INPUT -i eth0 -s 127.0.0.0/8 -j DROP # Loopback on publi

# Step 13: Prevent fragmentation attacks
sudo iptables -A INPUT -f -j DROP

# Step 14: ICMP rate limiting (allow ping but rate limit)
sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 5/s --
sudo iptables -A INPUT -p icmp -j DROP

echo "Attack prevention rules configured"

# PART 7: LOGGING
# Step 15: Log dropped packets (with rate limiting to prevent log flooding)

```



```
sudo iptables -A INPUT -m limit --limit 5/m -j LOG --log-prefix "[DROPPED] " --

echo "Logging configured"

# PART 8: FORWARDING (if acting as gateway)
# Step 16: Enable forwarding for internal network
sudo sysctl -w net.ipv4.ip_forward=1

# Step 17: Allow forwarding from internal network
sudo iptables -A FORWARD -i eth1 -s 192.168.1.0/24 -j ACCEPT
sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# Step 18: Masquerade outgoing traffic from internal network
sudo iptables -t nat -A POSTROUTING -o eth0 -s 192.168.1.0/24 -j MASQUERADE

echo "Forwarding and NAT configured"

# PART 9: VERIFICATION AND DOCUMENTATION
# Step 19: View complete ruleset
echo "=== COMPLETE FIREWALL CONFIGURATION ==="
echo ""
echo "DEFAULT POLICIES:"
sudo iptables -L | grep "policy"
echo ""
echo "INPUT RULES:"
sudo iptables -L INPUT -n -v | head -20
echo ""
echo "NAT RULES:"
sudo iptables -t nat -L -n -v
echo ""

# Step 20: Create configuration documentation
cat > /tmp/firewall-policy-doc.txt << 'EOF'
# FIREWALL SECURITY POLICY - Lab 15

## Architecture
- External Interface: eth0 (203.0.113.1)
- Internal Interface: eth1 (192.168.1.10)
- Internal Network: 192.168.1.0/24

## Default Policies
- INPUT: DROP (deny all by default)
- OUTPUT: ACCEPT (allow all outgoing)
- FORWARD: DROP (no forwarding by default, except internal)
```

Allowed Services

1. SSH (22/tcp) - From admin network (10.0.0.0/8) only
 - Rate limited: 5 connections per minute
 - Protects against brute force
2. HTTP (80/tcp) - From anywhere
 - Unlimited connections
 - For public web server
3. HTTPS (443/tcp) - From anywhere
 - Unlimited connections
 - For secure web traffic
4. MySQL (3306/tcp) - From internal network only
 - 192.168.1.0/24 only
 - Backend database access
5. PostgreSQL (5432/tcp) - From internal network only
 - 192.168.1.0/24 only
6. Redis (6379/tcp) - From internal network only
 - Cache server access

Attack Prevention

- SYN flood: Rate limited to 5/sec with burst 10
- Invalid packets: TCP flags sanity checks
- IP spoofing: Private IPs blocked on public interface
- Fragmentation: All fragmented packets dropped
- ICMP flood: Rate limited to 5/sec
- Brute force: SSH limited to 5 connections/min per IP

Forwarding Rules

- Internal network can reach external (via NAT masquerade)
- Bidirectional for established connections
- All new forward attempts dropped unless explicitly allowed

Logging

- All dropped packets logged with [DROPPED] prefix
- Log rate limited to 5 per minute to prevent flooding
- Logs go to /var/log/kern.log

Testing

See Lab 15 validation commands below

EOF

```
echo "Policy documentation created"
```

```
# Step 21: Save persistent configuration
```

```
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

```
# Step 22: Enable persistence on boot
```

```
sudo systemctl enable netfilter-persistent
```

```
echo "Configuration saved and persistence enabled"
```

```
echo ""
```

```
echo "=== LAB 15 CONFIGURATION COMPLETE ==="
```

Validation:

```
bash
```

```
# Test SSH access (should work from 10.0.0.0/8)
ssh -o ConnectTimeout=5 user@203.0.113.1
# From 10.x.x.x network: Should work
# From other networks: Should timeout

# Test HTTP access (should work from anywhere)
curl http://203.0.113.1
# Should work or show "Connection refused" if no web server

# Test HTTPS (should work from anywhere)
curl https://203.0.113.1
# Should work or show "Connection refused" if no web server

# Test internal network communication
ssh user@192.168.1.20
# Should work from 192.168.1.10 (internal network)

# View active connections
sudo conntrack -L | head -10
# Should show established connections

# View logging
sudo tail -f /var/log/kern.log | grep DROPPED
# In another terminal:
timeout 1 nc localhost 23
# Watch for log entry

# Test attack prevention
# FIN scan attempt:
sudo nmap -sF localhost
# Should be logged and dropped

# Verify forwarding works (if applicable)
# From 192.168.1.20 try reaching 203.0.113.1
ping 203.0.113.1
# Should work via FORWARD rules
```

Complete Configuration Checklist:

```
✓ Default policies set (DROP, ACCEPT, DROP)
✓ Loopback allowed
✓ Stateful filtering enabled
✓ SSH restricted and rate limited
✓ HTTP/HTTPS open
✓ Internal services protected
✓ Attack prevention active
✓ Logging configured
✓ Forwarding/NAT working
✓ Configuration persistent
✓ Documentation complete
```

Post-Lab Analysis:

1. Review the complete rule set
2. Identify potential improvements
3. Consider which rules could be more restrictive
4. Test the configuration against attack patterns

Challenge Exercises:

```
bash

# 1. Add DDoS protection for HTTP
# 2. Add VPN access (UDP 1194)
# 3. Add FTP with specific IPs only
# 4. Add logging rotation for large logs
# 5. Create backup/restore procedure
```

LEVEL 4: REAL-WORLD SCENARIOS (Labs 16-20)

LAB 16: Web Server Security Hardening

Objective: Firewall configuration for production web server

Difficulty: ★★★★★ Advanced

Time: 40 minutes

Scenario:

Production Web Server

- Ubuntu Server 20.04
- Running Apache + PHP + MySQL
- SSL/TLS for HTTPS
- Cron jobs for backups

Lab Steps:

```
bash
```

PART 1: BASELINE CONFIGURATION

```
sudo iptables -F
sudo iptables -P INPUT DROP
sudo iptables -P OUTPUT ACCEPT
sudo iptables -P FORWARD DROP
```

PART 2: ESSENTIAL RULES

Loopback

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

Stateful

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

PART 3: WEB SERVER SERVICES

HTTP

```
sudo iptables -A INPUT -p tcp --dport 80 -m state --state NEW -j ACCEPT
```

HTTPS (critical for modern web)

```
sudo iptables -A INPUT -p tcp --dport 443 -m state --state NEW -j ACCEPT
```

PART 4: ADMIN ACCESS

SSH restricted to admin IPs

```
sudo iptables -A INPUT -s 203.0.113.50/32 -p tcp --dport 22 -m state --state NEW -j ACCEPT
```

```
sudo iptables -A INPUT -s 203.0.113.51/32 -p tcp --dport 22 -m state --state NEW -j ACCEPT
```

Block all other SSH

```
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

PART 5: ATTACK PREVENTION FOR WEB

Rate limit new connections (prevent stress)

```
sudo iptables -A INPUT -p tcp --dport 80 -m limit --limit 100/s --limit-burst 200 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

```
sudo iptables -A INPUT -p tcp --dport 443 -m limit --limit 100/s --limit-burst 200 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 443 -j DROP
```

SYN flood protection

```
sudo iptables -A INPUT -p tcp --syn -m limit --limit 10/s --limit-burst 20 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --syn -j DROP
```

PART 6: INVALID TRAFFIC

```
sudo iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
```

```
sudo iptables -A INPUT -p tcp --tcp-flags ALL FIN -j DROP
```

```
sudo iptables -A INPUT -p tcp ! --syn -m conntrack --ctstate NEW -j DROP
```

```
# PART 7: ICMP (allow limited ping for monitoring)
sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 10/s -j ACCEPT
sudo iptables -A INPUT -p icmp -j DROP

# PART 8: OUTPUT CONTROL (web server should not initiate many connections)
# Allow outbound DNS (for domain lookups)
sudo iptables -A OUTPUT -p udp --dport 53 -j ACCEPT
sudo iptables -A OUTPUT -p tcp --dport 53 -j ACCEPT

# Allow NTP (time synchronization)
sudo iptables -A OUTPUT -p udp --dport 123 -j ACCEPT

# Allow outbound for package management updates
sudo iptables -A OUTPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A OUTPUT -p tcp --dport 443 -j ACCEPT

# PART 9: LOGGING
sudo iptables -A INPUT -m limit --limit 5/m -j LOG --log-prefix "[WEB-BLOCKED]"

# PART 10: PERSISTENCE
sudo iptables-save | sudo tee /etc/iptables/rules.v4
sudo systemctl enable netfilter-persistent

echo "Web server firewall configured"
```

Testing Protocol:

```
bash
```



```
# Test 1: HTTP/HTTPS works
curl http://localhost
curl https://localhost -k # -k ignores self-signed

# Test 2: SSH works from admin IPs
ssh -v root@localhost
# Should work if from 203.0.113.50

# Test 3: SSH blocked from other IPs
# From other IP:
ssh root@203.0.113.1
# Should timeout

# Test 4: Rate limiting works
for i in {1..150}; do curl http://localhost:80 & done
wait
# Some should fail after burst limit

# Test 5: No unexpected outbound
# Install tcpdump to monitor
sudo tcpdump -i eth0 -n "dst port not (53 or 123 or 80 or 443)"
# Should see minimal unexpected traffic
```

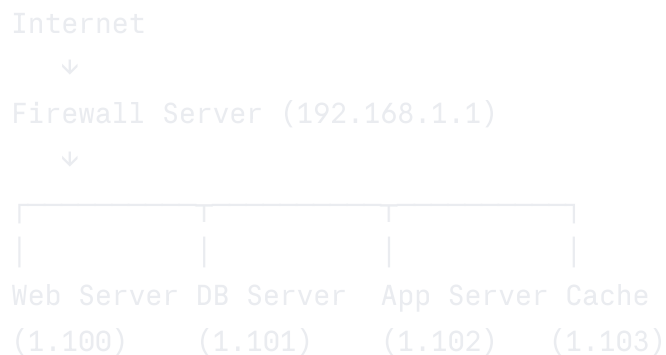
LAB 17: Multi-Server Network Firewall

Objective: Firewall for network with multiple servers

Difficulty: ★★★★★ Advanced

Time: 45 minutes

Network Diagram:



Lab Steps:

bash

```
# Enable IP forwarding (required for routing between networks)
```

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
# Reset firewall
```

```
sudo iptables -F
```

```
sudo iptables -P INPUT DROP
```

```
sudo iptables -P OUTPUT ACCEPT
```

```
sudo iptables -P FORWARD DROP
```

```
# PART 1: INTERNAL FORWARDING
```

```
# Allow web server to connect to database
```

```
sudo iptables -A FORWARD -s 192.168.1.100 -d 192.168.1.101 -p tcp --dport 3306
```

```
sudo iptables -A FORWARD -d 192.168.1.100 -s 192.168.1.101 -p tcp --sport 3306
```

```
# Allow app server to connect to cache
```

```
sudo iptables -A FORWARD -s 192.168.1.102 -d 192.168.1.103 -p tcp --dport 6379
```

```
sudo iptables -A FORWARD -d 192.168.1.102 -s 192.168.1.103 -p tcp --sport 6379
```

```
# Allow app server to database
```

```
sudo iptables -A FORWARD -s 192.168.1.102 -d 192.168.1.101 -p tcp --dport 3306
```

```
sudo iptables -A FORWARD -d 192.168.1.102 -s 192.168.1.101 -p tcp --sport 3306
```

```
# PART 2: EXTERNAL FORWARDING
```

```
# Only allow HTTP/HTTPS to web server
```

```
sudo iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 80 -d 192.168.1.100 -m
```

```
sudo iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 443 -d 192.168.1.100 -m
```

```
# Allow responses back
```

```
sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

```
# PART 3: OUTPUT RULES
```

```
# Servers need DNS and NTP
```

```
sudo iptables -A OUTPUT -d 192.168.1.0/24 -j ACCEPT
```

```
sudo iptables -A OUTPUT -p udp --dport 53 -j ACCEPT
```

```
sudo iptables -A OUTPUT -p tcp --dport 53 -j ACCEPT
```

```
sudo iptables -A OUTPUT -p udp --dport 123 -j ACCEPT
```

```
# PART 4: NAT FOR EXTERNAL TRAFFIC
```

```
sudo iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth0 -j MASQUERADE
```

```
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j DNAT --to-desti
```

```
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 443 -j DNAT --to-dest
```

```
# Save configuration
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

Validation:

```
bash

# From web server (192.168.1.100):
# Can reach database?
telnet 192.168.1.101 3306
# Should connect

# From app server (192.168.1.102):
# Can reach cache?
telnet 192.168.1.103 6379
# Should connect

# From internet:
curl http://203.0.113.1
# Should reach web server (192.168.1.100) via forwarding

# Database server can't be reached directly:
telnet 203.0.113.1 3306
# Should timeout
```

LAB 18: VPN and Remote Access

Objective: Secure firewall rules for VPN access

Difficulty: ★★★★★ Advanced

Time: 50 minutes

Prerequisites:

- OpenVPN installed and configured
- VPN clients (simulated on separate machine)

Lab Steps:

```
bash
```

```
# Setup VPN firewall rules

# Reset firewall
sudo iptables -F
sudo iptables -P INPUT DROP
sudo iptables -P OUTPUT ACCEPT
sudo iptables -P FORWARD DROP

# PART 1: VPN TRAFFIC
# OpenVPN UDP (1194)
sudo iptables -A INPUT -p udp --dport 1194 -j ACCEPT

# OpenVPN TCP (optional)
sudo iptables -A INPUT -p tcp --dport 1194 -j ACCEPT

# TUN interface (VPN virtual interface)
sudo iptables -A INPUT -i tun0 -j ACCEPT
sudo iptables -A OUTPUT -o tun0 -j ACCEPT
sudo iptables -A FORWARD -i tun0 -j ACCEPT
sudo iptables -A FORWARD -o tun0 -j ACCEPT

# PART 2: FORWARD VPN TRAFFIC TO INTERNAL NETWORK
# VPN clients → internal network
sudo iptables -A FORWARD -i tun0 -o eth1 -s 10.8.0.0/24 -d 192.168.1.0/24 -j ACCEPT
sudo iptables -A FORWARD -i eth1 -o tun0 -d 10.8.0.0/24 -s 192.168.1.0/24 -m conntrack --ctstate NEW -j ACCEPT

# PART 3: NAT FOR VPN CLIENTS
# VPN clients appear as if from firewall
sudo iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth1 -j MASQUERADE

# PART 4: LIMIT VPN TRAFFIC (optional, for bandwidth management)
# Max 10 Mbps per VPN connection
sudo iptables -A FORWARD -i tun0 -m limit --limit 1250/s --limit-burst 2500 -j ACCEPT

# Save configuration
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

LAB 19: DDoS Mitigation

Objective: Configure advanced DDoS protection rules

Difficulty: ★★★★★ Advanced

Time: 50 minutes

Lab Steps:

```
bash
```

```
# Comprehensive DDoS protection strategy
```

```
# Reset and set policies
```

```
sudo iptables -F
```

```
sudo iptables -P INPUT DROP
```

```
sudo iptables -P OUTPUT ACCEPT
```

```
sudo iptables -P FORWARD DROP
```

```
# PART 1: VOLUMETRIC ATTACK DEFENSE (UDP floods)
```

```
# Limit UDP to 100 pps (packets per second)
```

```
sudo iptables -A INPUT -p udp -m limit --limit 100/s --limit-burst 200 -j ACCEPT
```

```
sudo iptables -A INPUT -p udp -j DROP
```

```
# PART 2: PROTOCOL ATTACK DEFENSE (SYN, DNS, etc)
```

```
# SYN flood
```

```
sudo iptables -A INPUT -p tcp --syn -m limit --limit 20/s --limit-burst 40 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --syn -j DROP
```

```
# DNS query rate limiting
```

```
sudo iptables -A INPUT -p udp --dport 53 -m limit --limit 50/s --limit-burst 100 -j ACCEPT
```

```
sudo iptables -A INPUT -p udp --dport 53 -j DROP
```

```
# ICMP rate limiting
```

```
sudo iptables -A INPUT -p icmp -m limit --limit 10/s --limit-burst 20 -j ACCEPT
```

```
sudo iptables -A INPUT -p icmp -j DROP
```

```
# PART 3: APPLICATION LAYER (HTTP/HTTPS)
```

```
# Connection state tracking
```

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

```
# New HTTP/HTTPS connections with rate limiting
```

```
sudo iptables -A INPUT -p tcp --dport 80 -m limit --limit 200/s --limit-burst 400 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

```
sudo iptables -A INPUT -p tcp --dport 443 -m limit --limit 200/s --limit-burst 400 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 443 -j DROP
```

```
# PART 4: INVALID PACKET DEFENSE
```

```
# Drop malformed packets
```

```
sudo iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
```

```
sudo iptables -A INPUT -p tcp --tcp-flags ALL FIN -j DROP
```

```
sudo iptables -A INPUT -p tcp ! --syn -m conntrack --ctstate NEW -j DROP
```

```
# PART 5: PER-IP CONNECTION LIMITS
# Max 1000 concurrent connections per IP
sudo iptables -A INPUT -p tcp --syn -m connlimit --connlimit-above 1000 -j DROP

# Max 100 new connections per IP to port 80
sudo iptables -A INPUT -p tcp --dport 80 -m state --state NEW -m recent --set
sudo iptables -A INPUT -p tcp --dport 80 -m state --state NEW -m recent --update

# PART 6: GEO-IP BLOCKING (example)
# Block known bad countries (would need IP list)
# This is simplified - real implementation requires maintaining IP lists

# PART 7: LOGGING
sudo iptables -A INPUT -m limit --limit 2/m -j LOG --log-prefix "[DDoS-ATTEMPT]"

# PART 8: PERSISTENCE
sudo iptables-save | sudo tee /etc/iptables/rules.v4
sudo systemctl enable netfilter-persistent
```

DDoS Testing (Simulated):

```
bash

# WARNING: Only test on your own systems!

# Simulate SYN flood (minimal)
sudo hping3 -S --flood localhost -p 80 &
# Firewall should rate limit/drop

# Simulate UDP flood
sudo hping3 -2 --flood localhost -p 53 &
# Should be rate limited

# Monitor effectiveness
sudo iptables -L -n -v | grep "pkts"
# Higher packet count = more traffic being filtered
```

LAB 20: Monitoring and Incident Response

Objective: Setup firewall monitoring and incident response procedures

Difficulty: ★★★★★ Advanced

Time: 60 minutes

Lab Steps:

bash

PART 1: COMPREHENSIVE LOGGING SETUP

Enable kernel logging for all dropped packets

```
sudo iptables -A INPUT -j LOG --log-prefix "[FIREWALL-DROP] " --log-level 7 --1
```

PART 2: LOG ROTATION CONFIGURATION

```
sudo cat > /etc/logrotate.d/iptables << 'EOF'
```

```
/var/log/iptables.log {  
    daily  
    rotate 10  
    compress  
    delaycompress  
    missingok  
    notifempty  
    create 0640 syslog adm  
    sharedscripts  
    postrotate  
        /usr/lib/rsyslog/rsyslog-rotate  
    endscript  
}
```

EOF

PART 3: REAL-TIME MONITORING SCRIPT

```
sudo cat > /usr/local/bin/firewall-monitor.sh << 'EOF'
```

```
#!/bin/bash
```

Firewall Monitoring Script

Color definitions

```
RED='\033[0;31m'
```

```
YELLOW='\033[1;33m'
```

```
GREEN='\033[0;32m'
```

```
NC='\033[0m' # No Color
```

```
echo -e "${GREEN}=== FIREWALL STATUS MONITORING ===${NC}"
```

Check if UFW is running

```
echo -e "\n${YELLOW}UFW Status:${NC}"
```

```
sudo ufw status
```

Check iptables rules count

```
echo -e "\n${YELLOW}iptables Rules Count:${NC}"
```

```
echo "INPUT: $(sudo iptables -L INPUT -n | wc -l) lines"
```

```
echo "OUTPUT: $(sudo iptables -L OUTPUT -n | wc -l) lines"
```

```
echo "FORWARD: $(sudo iptables -L FORWARD -n | wc -l) lines"

# Show packet counts per rule (highest traffic)
echo -e "\n${YELLOW}Top Rules by Packet Count:${NC}"
sudo iptables -L -n -v | grep -v "pkts" | sort -k1 -rn | head -5

# Check for recent drops
echo -e "\n${YELLOW}Recent Dropped Packets (last 10):${NC}"
sudo tail -10 /var/log/kern.log | grep "FIREWALL-DROP" || echo "No recent drops"

# Check active connections
echo -e "\n${YELLOW}Active Connections:${NC}"
sudo conntrack -L 2>/dev/null | wc -l || echo "Conntrack not available"

# Check for potential attacks in logs
echo -e "\n${YELLOW}Potential Attack Indicators:${NC}"
echo "SYN floods (last 5 min):"
sudo grep "SYN" /var/log/kern.log | tail -5 | wc -l

echo "Port scans (last 5 min):"
sudo grep "FIREWALL-DROP.*tcp" /var/log/kern.log | tail -5 | wc -l

echo "UDP floods (last 5 min):"
sudo grep "FIREWALL-DROP.*udp" /var/log/kern.log | tail -5 | wc -l

echo -e "\n${GREEN}=== MONITORING COMPLETE ===${NC}"
EOF

sudo chmod +x /usr/local/bin/firewall-monitor.sh
```

PART 4: INCIDENT RESPONSE PROCEDURES

Create incident response script

```
sudo cat > /usr/local/bin/firewall-incident-response.sh << 'EOF'
#!/bin/bash
```

Incident Response Script

Usage: sudo firewall-incident-response.sh [block|analyze|remediate] [ip|port]

ACTION=\$1

TARGET=\$2

```
if [ -z "$ACTION" ] || [ -z "$TARGET" ]; then
    echo "Usage: $0 [block|analyze|remediate] [ip|port]"
```

```

    exit 1
fi

case $ACTION in
    block)
        echo "Blocking $TARGET..."
        iptables -A INPUT -s $TARGET -j DROP
        iptables -A INPUT -d $TARGET -j DROP
        echo "BLOCKED" >> /var/log/firewall-incidents.log
        ;;
    analyze)
        echo "Analyzing traffic from $TARGET..."
        grep $TARGET /var/log/kern.log | tail -20
        conntrack -L | grep $TARGET
        ;;
    remediate)
        echo "Removing block on $TARGET..."
        iptables -D INPUT -s $TARGET -j DROP
        iptables -D INPUT -d $TARGET -j DROP
        echo "UNBLOCKED" >> /var/log/firewall-incidents.log
        ;;
    *)
        echo "Unknown action"
        exit 1
        ;;
esac
EOF

```

```

sudo chmod +x /usr/local/bin/firewall-incident-response.sh

```

PART 5: AUTOMATED ALERT SYSTEM

```

# Create alert script for high traffic

```

```

sudo cat > /usr/local/bin/firewall-alerts.sh << 'EOF'
#!/bin/bash

```

```

# Monitor for unusual patterns and send alerts

```

```

THRESHOLD_DROPS=100 # Drops per minute
THRESHOLD_CONNECTIONS=1000 # Active connections

```

```

# Check dropped packets rate
DROPS=$(sudo tail -1000 /var/log/kern.log | grep -c "FIREWALL-DROP")

```

```
if [ $DROPS -gt $THRESHOLD_DROPS ]; then
    echo "ALERT: High drop rate ($DROPS/sample)"
    # Could send email, log to monitoring system, etc.
fi

# Check connection count
CONNS=$(sudo conntrack -L 2>/dev/null | wc -l)

if [ $CONNS -gt $THRESHOLD_CONNECTIONS ]; then
    echo "ALERT: High connection count ($CONNS)"
fi
EOF

sudo chmod +x /usr/local/bin/firewall-alerts.sh

# PART 6: SETUP CRON JOBS FOR MONITORING

# Monitor every 5 minutes
(crontab -l 2>/dev/null; echo "*/5 * * * * /usr/local/bin/firewall-alerts.sh")

# Analyze logs daily at 2 AM
(crontab -l 2>/dev/null; echo "0 2 * * * /usr/local/bin/firewall-monitor.sh >>")

# PART 7: TEST THE MONITORING
echo "Running firewall monitor..."
sudo /usr/local/bin/firewall-monitor.sh

# PART 8: INCIDENT LOG SETUP
sudo touch /var/log/firewall-incidents.log
sudo chmod 640 /var/log/firewall-incidents.log

echo "Monitoring and incident response setup complete!"
```

Monitoring Checklist:

Daily Checks:

- Review firewall logs for unusual patterns
- Check connection count trends
- Monitor CPU/memory usage
- Verify firewall rules still in place

Weekly Checks:

- Analyze traffic patterns
- Review dropped packet statistics
- Check for new attack patterns
- Update threat intelligence

Monthly Checks:

- Full security audit
- Firewall rule review and optimization
- Backup configuration
- Test incident response procedures

Incident Response Procedures:

bash

Immediate response to attack:

1. Block attacker IP:

```
sudo firewall-incident-response.sh block [attacker-ip]
```

2. Analyze traffic:

```
sudo firewall-incident-response.sh analyze [attacker-ip]
```

3. Enable enhanced logging:

```
sudo iptables -A INPUT -s [attacker-ip] -j LOG --log-prefix "[ATTACK] "
```

4. Collect evidence:

```
sudo tcpdump -i eth0 -s 0 -n -w /tmp/capture.pcap 'src [attacker-ip]'
```

5. Escalate if needed:

- Contact ISP/hosting provider
- Report to CERT/authorities
- Alert security team

6. Remediate:

```
sudo firewall-incident-response.sh remediate [attacker-ip]
```

Post-Incident Review:

- Document:
- Time and duration of incident
 - Attack type and characteristics
 - Response actions taken
 - Effectiveness of response
 - Improvements needed
 - Updated firewall rules

 SUMMARY TABLE: All 20 Labs

Lab	Title	Difficulty	Time	Key Skills
1	UFW Installation	★	10m	Install, enable, verify
2	Allow/Deny Ports	★	15m	Basic rules, port numbers
3	IP-based Rules	★	20m	CIDR notation, source/dest
4	Rate Limiting	★★	20m	DDoS protection, burst
5	Rule Management	★★	15m	Delete, insert, modify
6	iptables Setup	★★	15m	Policies, persistence
7	Adding Rules	★★	20m	Multiple protocols, flags
8	Deleting Rules	★★	20m	Line numbers, safe deletion
9	Advanced Matching	★★★	25m	Multiport, ranges, limits
10	Logging	★★★	20m	Log levels, monitoring
11	UFW+iptables	★★★	25m	Coexistence, interaction
12	NAT/Forwarding	★★★★	30m	DNAT, SNAT, masquerade
13	Connection Tracking	★★★★	25m	State matching, conntrack
14	Attack Prevention	★★★★	30m	Floods, scans, spoofing
15	Complete Policy	★★★★★	45m	Production-ready config

Lab	Title	Difficulty	Time	Key Skills
16	Web Server	★★★★★	40m	HTTPS, rate limiting
17	Multi-Server	★★★★★	45m	Forwarding, segmentation
18	VPN Access	★★★★★★	50m	VPN rules, TUN interface
19	DDoS Mitigation	★★★★★★	50m	Advanced rate limiting
20	Monitoring	★★★★★★	60m	Logs, alerts, automation

Learning Path Recommendations

For Beginners (Focus: Labs 1-7)

Weeks 1-2: Master UFW and basic iptables

- Start with Lab 1-5 (UFW basics)
- Understand firewall concepts
- Practice daily

Weeks 3-4: Learn iptables fundamentals

- Complete Lab 6-7 (iptables installation and rules)
- Compare UFW and iptables
- Build confidence with rules

For Intermediate (Focus: Labs 8-12)

Weeks 5-6: Advanced iptables and coexistence

- Lab 8-12 (rule management, advanced features, UFW+iptables)
- Understand NAT/forwarding
- Practice troubleshooting

Weeks 7-8: Specialized topics

- Lab 13 (connection tracking)
- Lab 14 (attack prevention)
- Real-world scenarios

For Advanced (Focus: Labs 13-20)

Weeks 9-10: Security hardening

- Lab 15 (complete policies)
- Lab 16 (production web server)
- Lab 17 (network architecture)

Weeks 11-12: Specialized scenarios

- Lab 18-19 (VPN, DDoS)
- Lab 20 (monitoring, automation)
- Build incident response procedures

Quick Reference Commands

UFW Quick Commands

```
bash

sudo ufw enable           # Enable UFW
sudo ufw disable          # Disable UFW
sudo ufw allow 22         # Allow port 22
sudo ufw deny 23          # Deny port 23
sudo ufw allow from 192.168.1.100 to any port 22 # IP-based rule
sudo ufw status numbered  # View rules with numbers
sudo ufw delete 1         # Delete rule #1
sudo ufw limit 22/tcp     # Rate limit SSH
sudo ufw reset            # Delete all rules
```

iptables Quick Commands

```
bash
```

```
sudo iptables -L -n --line-numbers      # List all rules
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT # Add rule
sudo iptables -D INPUT 1                # Delete rule #1
sudo iptables -P INPUT DROP             # Set policy
sudo iptables -F                        # Flush all rules
sudo iptables-save > /tmp/backup.rules   # Backup rules
sudo iptables-restore < /tmp/backup.rules # Restore rules
sudo iptables -t nat -L -n              # View NAT rules
```

Created: 2026 | **For:** Kali Linux / Ubuntu **Level:** Beginner to Advanced | **Total Labs:** 20